Automated Vulnerability Discovery & Remediation Pipeline

Author: DevSecOps Lab

**Date

Executive Summary

This project implements a shift-left DevSecOps workflow for a containerized WordPress deployment. We deployed an intentionally outdated baseline, automated discovery with WPScan in GitHub Actions, remediated through core upgrades and container hardening, published a fixed image to GitHub Container Registry (GHCR) and Docker Hub, and verified improvements via automated rescanning.

Outcome: WordPress core upgraded from **5.8.3 ? 6.7.2**, known CVE exposure reduced from **14 reported issues ? 0**, user enumeration blocked, debug mode disabled, and Apache/PHP attack surface reduced.

---

1. Environment Setup

1.1 Steps Taken

1. Created `docker/docker-compose.yml` with WordPress **5.8.3** and MySQL **5.7** (lab baseline).

2. Sta

1.2 Challenges & Solutions

| Challenge | Solution |

|----

---

2. Findings Overview

2.1 Key Vulnerabilities (Baseline)

| Finding | Risk | CVSS (indicative) |

|----

2.2 Risk Assessment

- **Likelihood:** High ? WordPress 5.8.x is actively targeted; public exploits exist for multiple listed CVEs.

- **Im

2.3 Exploitation Paths

1. **Authenticated SQL injection (WP < 6.0.3):** Attacker with contributor+ role crafts malicious post meta to extract `wp_users.user_pass` hashes ? offline crack ? admin takeover.

2. **U
force

Evidence: `scans/before-remediation-20250520T120000Z.txt`

---

3. Remediation Steps

3.1 Patching & Updating

| Component | Before | After |

|---

3.2 Hardening Measures

- **Non-root runtime:** Container runs as `www-data`; Apache listens on **8080**.

- **Pa

3.3 Before/After Scan Evidence

| Metric | Before | After |

|---

Files: `scans/before-remediation-*` vs `scans/after-remediation-*`

---

4. Fixed Image Build

| Resource | URL (replace with your org) |

|---

Build locally:

docker build -f dockerfiles/Dockerfile.hardened -t YOUR_USER/wordpress-hardened:latest .

docke

---

5. Tooling Justification

| Tool | Role | Why |

|---

---

6. DevSecOps Strategy (Shift-Left)

Security moves **left** because:

1. Scans run **before production** on every `main` push.

2. Fin

Appendix A ? Workflow Commands

Trigger scan workflow

gh wo

Trigger build + rescan

gh wo

Appendix B ? Commit History (Remediation Narrative)

1. `feat: initial vulnerable WordPress docker-compose deployment`

2. `ci:

End of report